

MPCA Module 1 — 5 Mark Questions & Answers

Q1. Explain the Von Neumann model with a neat labeled diagram.

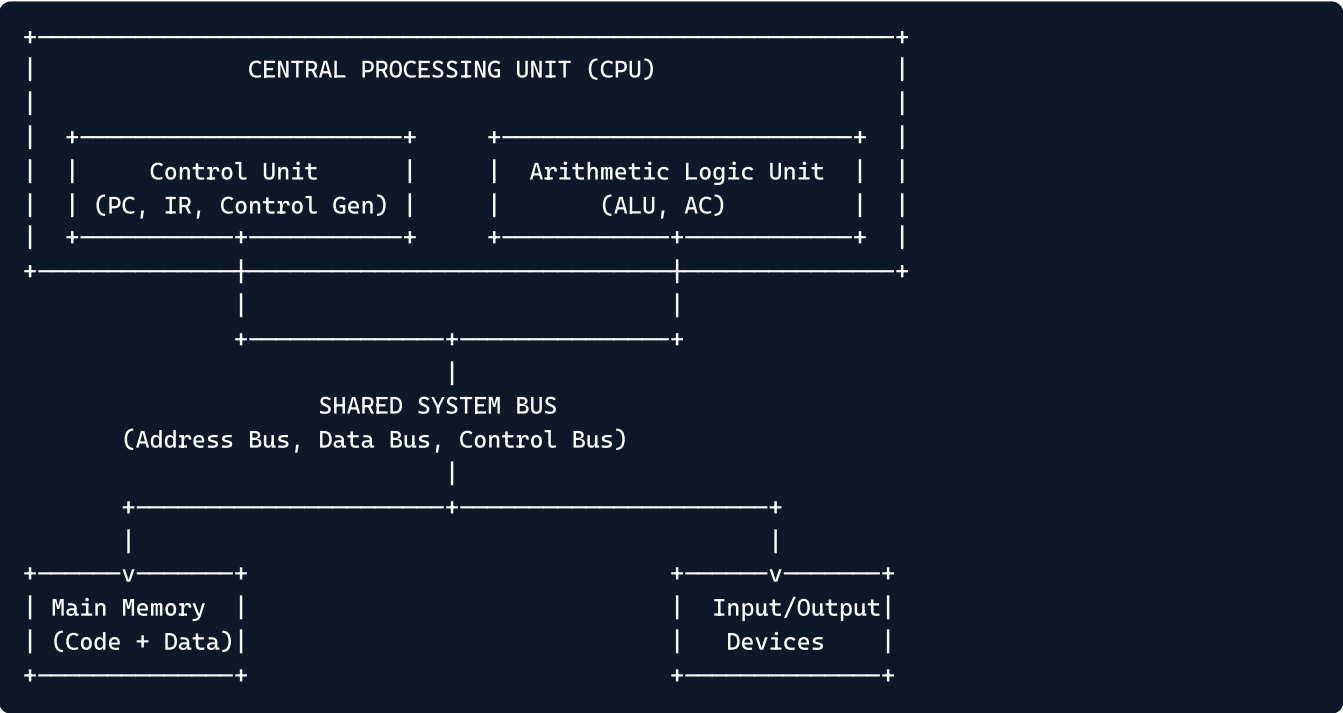
Introduction

The **Von Neumann Architecture** proposed by John von Neumann in 1945 forms the foundational model for modern general-purpose digital computers. Its defining characteristic is the **Stored Program Concept**, where both data and program instructions share the same memory space.

Key Components

- 1. **Central Processing Unit (CPU):**
 - * **Control Unit (CU):** Fetches instructions from memory, decodes them, and issues control signals.
 - * **Arithmetic Logic Unit (ALU):** Performs integer arithmetic (+, -, ×) and bitwise logical operations.
 - * **Registers:** High-speed storage locations inside CPU (Accumulator, Program Counter, Instruction Register).
- 2. **Main Memory (RAM):** Stores both data and instructions in binary form. Organized as addressable bytes.
- 3. **Shared System Bus:** Single set of wires (Address, Data, Control bus) connecting CPU, Memory, and I/O.
- 4. **Input/Output (I/O) System:** Interfaces the computer with external devices.

Block Diagram



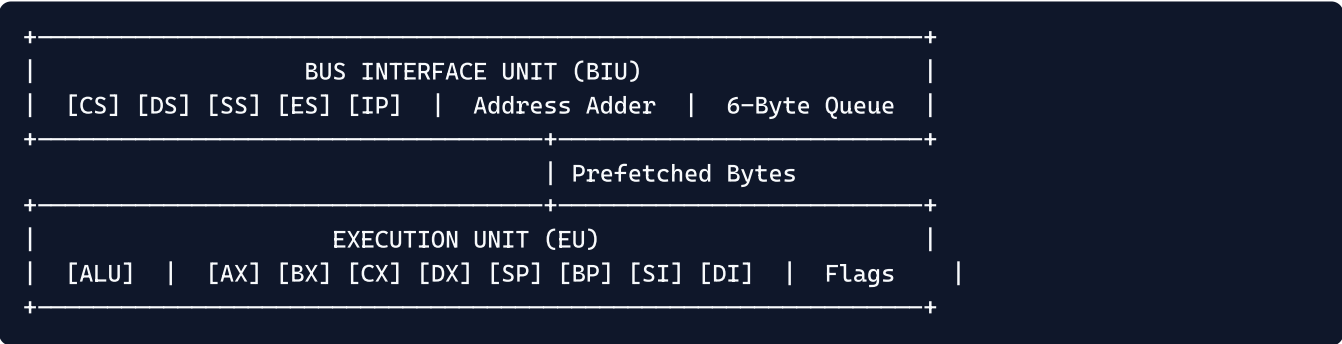
Q2. Compare Von Neumann and Harvard architecture (tabular format, at least 5 points).

Feature	Von Neumann Architecture	Harvard Architecture
Memory Structure	Single shared memory for both code and data	Separate physical memory units for code and data
Bus System	Single shared data/address bus	Separate instruction bus and data bus
Pipelining / Execution	Cannot fetch instruction and read/write data simultaneously	Can fetch instruction and read/write data simultaneously
Speed / Performance	Slower due to Von Neumann bottleneck	Faster execution due to parallel bus transfers
Hardware Complexity	Simpler hardware layout; cheaper to build	More complex hardware; higher cost due to dual buses
Memory Utilization	High; unallocated space usable by code or data	Strict allocation; unused code memory cannot store data
Application	General-purpose PCs, laptops, servers	DSPs (Digital Signal Processors), microcontrollers (PIC, ARM)

Q3. Explain the architecture of the 8086 microprocessor with a block diagram (BIU and EU in detail).

Introduction

The Intel 8086 is a 16-bit microprocessor with a 20-bit address bus (1 MB memory). To maximize efficiency, its internal architecture is divided into two independent, asynchronously operating units: the **Bus Interface Unit (BIU)** and the **Execution Unit (EU)**.



1. Bus Interface Unit (BIU)

- **Functions:** Handles all external bus transfers — instruction fetching, memory read/write, and I/O access.
- **Components:**
- **Segment Registers:** CS, DS, SS, ES (16-bit base segment addresses).
- **Instruction Pointer (IP):** Holds offset of next instruction.
- **20-bit Address Adder:** Calculates `Physical Address = Segment × 16 + Offset`.
- **6-Byte Instruction Queue:** Prefetches up to 6 bytes of instruction stream to enable basic pipelining.

2. Execution Unit (EU)

- **Functions:** Decodes and executes instructions fetched from the BIU queue.

- **Components:**
 - **Control Circuitry:** Decodes instructions into internal micro-operations.
 - **16-bit ALU:** Performs arithmetic and logical operations.
 - **General Purpose Registers:** AX, BX, CX, DX (8/16-bit accessible).
 - **Pointer & Index Registers:** SP, BP, SI, DI.
 - **Flag Register:** 16-bit register holding 9 active flags (6 Status, 3 Control).
-

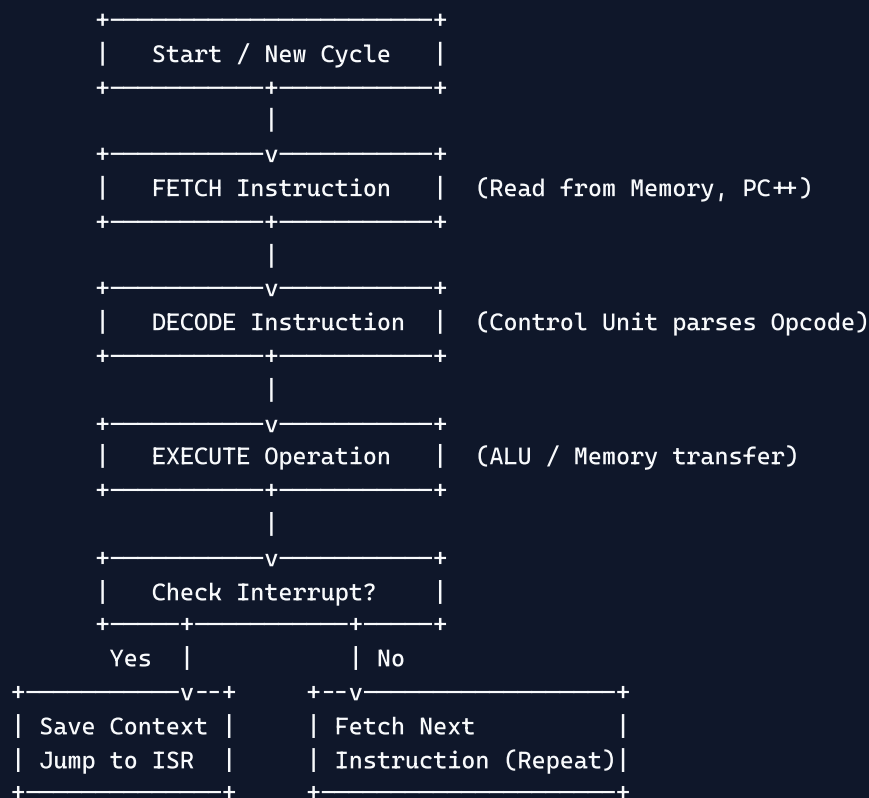
Q4. Explain any five performance measures of computer architecture with formulas.

1. **CPU Execution Time:** The total time taken by the CPU to execute a program.
 * $\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate}$
 2. **Clock Rate (Frequency):** The number of clock cycles produced by the processor oscillator per second (measured in Hz / GHz).
 * $\text{Clock Rate} = 1 / \text{Clock Cycle Time}$
 3. **CPI (Cycles Per Instruction):** The average number of clock cycles required per instruction.
 * $\text{CPI} = \text{Total Clock Cycles} / \text{Instruction Count}$
 4. **MIPS (Millions of Instructions Per Second):** Rate of instruction execution.
 * $\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6) = \text{Clock Rate} / (\text{CPI} \times 10^6)$
 5. **Throughput:** Total number of tasks completed per unit time.
 * $\text{Throughput} = \text{Executed Tasks} / \text{Total Time}$
-

Q5. Explain the instruction cycle of the Von Neumann model with a flow diagram, including interrupt handling.

Instruction Cycle Stages

1. **Fetch:** Read instruction from memory at address given by Program Counter (PC). Load into Instruction Register (IR). Increment PC.
2. **Decode:** Control Unit decodes opcode in IR, determines required operands.
3. **Execute:** ALU executes the instruction operation.
4. **Interrupt Check:** Check if an external hardware interrupt is pending.
 * If **Yes:** Push PC & Flags to stack, jump to Interrupt Service Routine (ISR).
 * If **No:** Proceed to next instruction fetch.



Q6. Classify and explain the different categories of the 8086 instruction set with examples.

- Data Transfer Instructions:** Move data between registers, memory, and I/O ports.
 * Examples: `MOV AX, BX`, `PUSH CX`, `POP DX`, `LEA SI, [BX]`, `XCHG AL, AH`.
- Arithmetic Instructions:** Perform binary/BCD arithmetic.
 * Examples: `ADD AX, 1000H`, `SUB BX, CX`, `INC SI`, `DEC DI`, `MUL BL`.
- Logical & Bit Manipulation Instructions:** Perform bitwise logic and shifts.
 * Examples: `AND AL, 0FH`, `OR BL, CL`, `XOR AX, AX`, `SHL BX, 1`, `ROL AL, 1`.
- Control Transfer (Branch) Instructions:** Alter execution sequence.
 * Examples: `JMP label`, `JE target`, `CALL subroutine`, `RET`, `LOOP start`.
- Processor Control Instructions:** Set/clear CPU flags and control execution states.
 * Examples: `STC` (Set Carry), `CLC` (Clear Carry), `STI` (Enable Int), `NOP`, `HLT`.

Q7. Explain any five addressing modes of the 8086 with syntax, effective address calculation, and examples.

- Immediate Addressing Mode:** Operand is a constant embedded in the instruction.
 * Syntax: `MOV reg, immediate_value`
 * Example: `MOV AX, 1234H` (No EA calculation; value = `1234H`).
- Register Addressing Mode:** Operand resides in an internal CPU register.
 * Syntax: `MOV reg1, reg2`
 * Example: `MOV AX, BX` (No EA calculation; data in `BX`).
- Direct Addressing Mode:** Effective address (16-bit offset) is given explicitly in instruction.
 * EA Formula: `EA = displacement`

* Example: `MOV AX, [2000H]` ($EA = 2000H$, $PA = DS \times 16 + 2000H$).

4. **Register Indirect Addressing Mode:** EA is contained in a base/index register (`BX` , `BP` , `SI` , `DI`).

* EA Formula: $EA = [BX / BP / SI / DI]$

* Example: `MOV AX, [BX]` ($EA = BX$, $PA = DS \times 16 + BX$).

5. **Base-Indexed Addressing Mode:** EA is the sum of a base register and an index register.

* EA Formula: $EA = Base_Reg + Index_Reg$

* Example: `MOV AX, [BX + SI]` ($EA = BX + SI$, $PA = DS \times 16 + (BX + SI)$).

Q8. Explain data transfer instructions (MOV, XCHG, LEA, PUSH, POP) with examples.

1. **MOV (Move):** Copies data from source to destination operand.

* Example: `MOV AX, BX` (Copies 16-bit value from `BX` into `AX`).

2. **XCHG (Exchange):** Swaps contents of source and destination.

* Example: `XCHG AL, AH` (Swaps higher and lower bytes of accumulator).

3. **LEA (Load Effective Address):** Computes 16-bit offset of source and loads it into a destination register.

* Example: `LEA SI, [BX + 2000H]` (Loads offset `BX + 2000H` into `SI`).

4. **PUSH:** Pushes a 16-bit register/memory value onto the top of the stack.

* Example: `PUSH AX` ($SP = SP - 2$, stores `AX` at `SS:SP`).

5. **POP:** Pops a 16-bit value from the top of the stack into a destination.

* Example: `POP CX` (Reads from `SS:SP` into `CX` , $SP = SP + 2$).

Q9. Explain control transfer instructions (JMP, CALL, RET, LOOP, Jcc) with examples.

1. **JMP (Unconditional Jump):** Transfers control unconditionally to target address.

* Example: `JMP AGAIN` (Sets $IP = \text{offset of } AGAIN$).

2. **CALL (Call Subroutine):** Invokes a function by saving return address onto stack.

* Example: `CALL MY_FUNC` (Pushes IP , sets $IP = \text{offset of } MY_FUNC$).

3. **RET (Return from Subroutine):** Restores return address from stack to resume main program.

* Example: `RET` (Pops stack into IP).

4. **LOOP (Loop Control):** Decrements `CX` ; jumps to target if $CX \neq 0$.

* Example: `LOOP BACK` ($CX = CX - 1$; if $CX \neq 0$ jump to `BACK`).

5. **Jcc (Conditional Jump):** Jumps only if specific flag condition is met.

* Example: `JZ TARGET` (Jump if Zero Flag $ZF = 1$), `JNC TARGET` (Jump if Carry Flag $CF = 0$).

Q10. Explain logical instructions (AND, OR, XOR, NOT, TEST, CMP) with the flags affected.

1. **AND:** Bitwise logical AND. Clears CF and OF. Sets ZF, SF, PF based on result.

* Example: `AND AL, 0FH` (Masks upper 4 bits of `AL` to 0).

2. **OR:** Bitwise logical OR. Clears CF and OF. Sets ZF, SF, PF.

* Example: `OR BL, 80H` (Sets MSB of `BL` to 1).

3. **XOR:** Bitwise logical XOR. Clears CF and OF. Sets ZF, SF, PF.

* Example: `XOR AX, AX` (Clears `AX` to 0 efficiently).

4. **NOT:** Bitwise 1's complement. **No flags affected.**

* Example: `NOT CX` (Inverts all bits of `CX`).

5. **TEST:** Performs non-destructive AND (result not saved). Sets ZF, SF, PF.

* *Example:* `TEST AL, 01H` (Checks if LSB is set without altering AL).

6. **CMP (Compare):** Subtracts source from destination non-destructively. Affects CF, ZF, SF, OF, AF, PF.

* *Example:* `CMP AX, BX` (Compares AX and BX; sets ZF=1 if equal, CF=1 if AX < BX).

Q11. Given clock rate, CPI, and instruction count, calculate CPU execution time and MIPS (numerical problem).

Problem Statement

A processor runs at a **Clock Rate of 2 GHz** ($2 \times 10^9 \text{ Hz}$). A program executes **10×10^6 instructions (Instruction Count)** with an average **CPI of 2.5**.

Calculate:

1. CPU Execution Time
2. MIPS rating of the system

Solution

1. Calculate CPU Execution Time:

$$\text{CPU Execution Time} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

$$\text{CPU Execution Time} = \frac{(10 \times 10^6) \times 2.5}{2 \times 10^9}$$

$$\text{CPU Execution Time} = \frac{25 \times 10^6}{2 \times 10^9} = 0.0125 \text{ seconds} \quad (12.5 \text{ ms})$$

2. Calculate MIPS Rating:

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{Execution Time} \times 10^6}$$

$$\text{MIPS} = \frac{10 \times 10^6}{0.0125 \times 10^6} = \frac{10}{0.0125} = 800 \text{ MIPS}$$

Alternatively:

$$\text{MIPS} = \frac{\text{Clock Rate}}{\text{CPI} \times 10^6} = \frac{2 \times 10^9}{2.5 \times 10^6} = 800 \text{ MIPS}$$
